

ChargeGrid SmartHub

Sistema Inteligente de Gerenciamento de Recarga

Documento Técnico — Sprint 2

Disciplina: Data Structure and Algorithms **Professor:** Erick Toshio Yamamoto **Curso:** Ciência da Computação — 1º Ano · 2026.1 **Instituição:** FIAP **Parceria:** GoodWe Brasil — EV Challenge 2026

Integrantes do grupo:

Nome	RM
João Pedro Quagliano	570233
Leticia Aiko Okano	571988
Thiago Calazans Luz Nakano	569151
Enzo Scattolin Furtado	570824
Guilherme De Lucena Fontes	569658
Matheus Levi Dagel	571961

Repositório: <https://github.com/JP-Quagliano/Sprint-02---DSA> **Data de entrega:** 16 de junho de 2026

Sumário Executivo

Este documento descreve a arquitetura, os algoritmos e a implementação do **ChargeGrid SmartHub**, um sistema em linguagem C que simula o gerenciamento operacional de uma rede de eletropostos comerciais GoodWe. O sistema cobre os quatro pilares funcionais do produto: controle de demanda elétrica, comunicação via protocolos abertos (OCPP 1.6), tarifação dinâmica e relatórios operacionais consolidados.

A implementação é composta por **seis módulos** independentes em C (linguagem padrão C11), com compilação multi-arquivo via `gcc`. O sistema gerencia até 20 sessões simultâneas, aplica algoritmo de *load balancing* por prioridade quando a demanda excede o limite contratado, calcula tarifa dinâmica em função de horário, ocupação e modelo de carregador, simula o tráfego OCPP 1.6-J entre o Charge Point e o Central System, e gera relatórios operacionais em arquivo `.txt` auditável.

O sistema foi projetado como a **camada de orquestração** do produto ChargeGrid SmartHub, sendo desenvolvido em paralelo com as camadas conversacional (disciplina Prompt and Artificial Intelligence) e de modelagem operacional (disciplina Pensamento Computacional e Automação com Python). A coerência cross-disciplinar é uma característica explícita do design.

1. Contexto e Problema

1.1 O EV Challenge 2026

O EV Challenge 2026 é um desafio integrador entre a FIAP e a GoodWe Brasil, na qual estudantes de Ciência da Computação desenvolvem soluções para a infraestrutura de carregamento de veículos elétricos. Nosso grupo escolheu o **Contexto A — ChargeGrid Intelligence**, voltado a operadores comerciais de eletropostos públicos e semipúblicos (shoppings, postos de combustível, estacionamentos, supermercados, frotas corporativas).

1.2 O produto ChargeGrid SmartHub

O **ChargeGrid SmartHub** é a plataforma de orquestração da rede comercial GoodWe. Ele atua sobre a linha de eletropostos **HCA G2**, composta por três modelos:

Modelo	Potência nominal	Caso de uso típico
GW7K-HCA-20	7 kW	Vagas de longa permanência (escritórios, shoppings)
GW11K-HCA-20	11 kW	Vagas mistas (supermercados, frotas leves)
GW22K-HCA-20	22 kW	Vagas de fluxo rápido (postos, frota corporativa)

A plataforma resolve quatro problemas operacionais centrais:

- 1. **Controle de demanda** — distribuir energia entre múltiplos pontos de recarga sem estourar a demanda contratada do edifício
- 2. **Comunicação aberta** — integrar carregadores ao sistema central via protocolo padrão (OCPP 1.6)
- 3. **Tarifação dinâmica** — precificar a recarga conforme horário, ocupação e modelo
- 4. **Inteligência operacional** — gerar relatórios e KPIs para a tomada de decisão do gestor

1.3 Persona atendida

O sistema atende o **operador comercial** — gestor responsável pela operação diária da estação. Tipicamente o gerente de um shopping ou supermercado, ou o supervisor de frota de uma transportadora. Suas necessidades incluem visibilidade sobre o status das sessões, garantia de que o limite elétrico do edifício não será violado, e capacidade de gerar relatórios de receita e energia para a diretoria.

1.4 Integração cross-disciplinar

Este sistema é a **camada de orquestração** do mesmo produto que aparece em outras duas disciplinas do challenge:

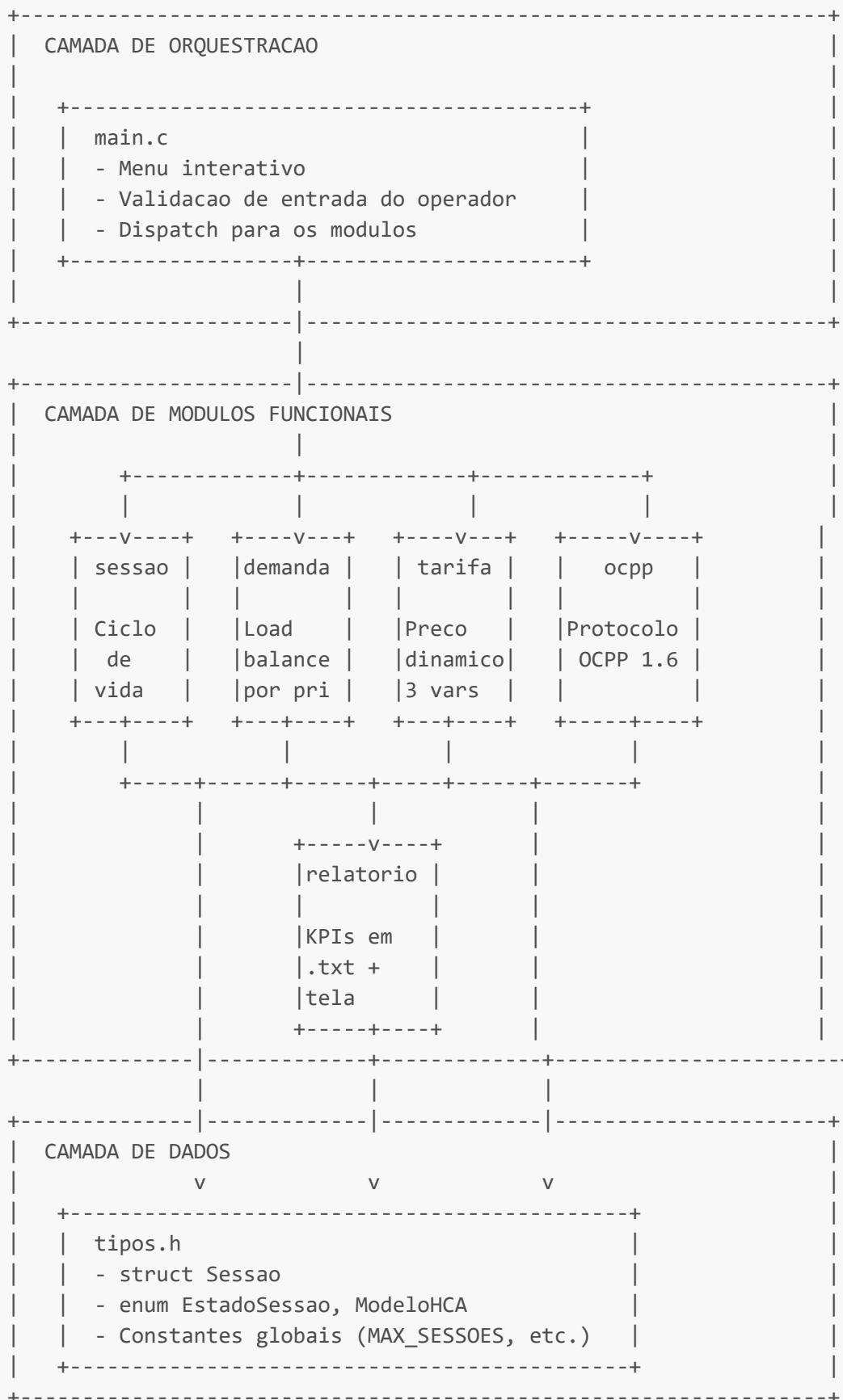
Disciplina	Camada do produto	Tecnologia
Prompt and Artificial Intelligence	Conversacional (SmartHub Assistant)	Python + LangChain + Gemini
Pensamento Computacional e Automação com Python	Modelagem operacional	Python + análise de dados
Data Structure and Algorithms	Orquestração	C (este documento)

Essa integração garante que o trabalho do grupo represente um **produto único** com três camadas tecnológicas — não três exercícios acadêmicos isolados.

2. Arquitetura

2.1 Visão geral

O sistema segue uma arquitetura em três camadas:



2.2 Lista de arquivos

```

dsa-sprint2/
├── .vscode/
│   └── tasks.json          # configuracao de compilacao do VS Code
├── src/
│   ├── tipos.h             # tipos e constantes compartilhadas
│   ├── sessao.h / .c       # gerenciamento de sessoes de recarga
│   ├── demanda.h / .c     # algoritmo de controle de demanda
│   ├── tarifa.h / .c      # calculo de tarifa dinamica
│   ├── ocpp.h / .c        # simulacao do protocolo OCPP 1.6
│   ├── relatorio.h / .c   # geracao de relatorios consolidados
│   └── main.c              # ponto de entrada e menu CLI
├── relatorios/             # destino dos arquivos .txt gerados
├── docs/
│   └── documento_tecnico.md
└── chargegrid.exe          # executavel apos compilacao

```

2.3 Padrão de includes

Adotamos o padrão de programação modular em C, com separação rigorosa entre interface (`.h`) e implementação (`.c`). A regra que garante ausência de dependências circulares é simples:

- Cada `.h` inclui apenas `tipos.h` (a fonte única dos tipos compartilhados)
- Os `.c` incluem livremente os `.h` de que precisam
- Nenhum `.h` inclui outro `.h` que não seja `tipos.h`

Esse padrão evita o erro comum de "redefinição de tipo" que aparece quando dois headers tentam declarar a mesma struct.

2.4 Decisões arquiteturais e justificativas

Array fixo em vez de alocação dinâmica

Optamos por `Sessao sessoes[MAX_SESSOES]` (array de tamanho fixo, `MAX_SESSOES = 20`) em vez de uma lista encadeada com `malloc/free`. Justificativa:

1. O contexto operacional (eletropostos comerciais) tem capacidade física limitada — 20 carregadores cobre o tamanho típico de uma estação de shopping
2. Elimina a necessidade de gerenciar memória manualmente, reduzindo a superfície de bugs (vazamentos, double-free, ponteiros pendurados)
3. Acesso $O(1)$ por índice é mais rápido que travessia de lista encadeada
4. Atende plenamente o critério "Controle organizado de sessões" da rubrica

A escolha é deliberada e adequada ao problema. Uma lista encadeada seria preferível se a capacidade fosse desconhecida em tempo de compilação — não é o caso aqui.

Variáveis globais com `extern`

O array `sessoes[]` e o relógio simulado (`hora_simulada`, `minuto_simulada`) são declarados como `extern` em `sessao.h` e definidos em `sessao.c`. Outros módulos acessam essas variáveis incluindo o header. Esse

padrão (declarar em `.h`, definir em `.c`) é o jeito canônico em C de compartilhar estado entre arquivos sem cair em redefinições.

Funções `static`

Funções auxiliares internas de cada módulo (como `busca_indice_por_id` em `sessao.c` ou `multiplicador_horario` em `tarifa.c`) são declaradas `static`, o que limita sua visibilidade ao próprio arquivo. É o equivalente em C a "privado" em outras linguagens: nenhum outro módulo consegue chamá-las acidentalmente.

3. Detalhamento dos módulos

3.1 `tipos.h` — definições compartilhadas

Fonte única de tipos e constantes do projeto. Não contém lógica. Define:

Constantes:

- `MAX_SESSOES` — capacidade do array de sessões (20)
- `DEMANDA_CONTRATADA` — limite elétrico do edifício (75.0 kW)
- `MIN_POTENCIA_CARGA` — limiar mínimo para carregar (1.4 kW, padrão IEC 61851)
- `TARIFA_BASE` — preço de referência por kWh (R\$ 0.85)

Enumerações:

- `EstadoSessao` — `OCIOSO`, `AGUARDANDO_AUTORIZACAO`, `CARREGANDO`, `PAUSADA_DEMANDA`, `FINALIZADA`, `ERRO`
- `ModeloHCA` — `GW7K`, `GW11K`, `GW22K`

Estrutura principal:

```
typedef struct {
    int      id;
    char     placa_veiculo[TAMANHO_PLACA];
    char     id_carregador[TAMANHO_ID_CARREG];
    ModeloHCA modelo;
    EstadoSessao estado;
    int      hora_inicio;
    int      minuto_inicio;
    float    potencia_nominal_kw;
    float    potencia_atual_kw;
    float    kwh_entregue;
    float    custo_acumulado;
    int      prioridade;
} Sessao;
```

A struct agrupa todos os dados de uma sessão num único registro. Distinção importante entre `potencia_nominal_kw` (capacidade do carregador) e `potencia_atual_kw` (potência realmente entregue, possivelmente reduzida pelo controle de demanda).

3.2 Módulo **sessao** — ciclo de vida das recargas

Responsabilidade: criar, listar, encerrar e simular passagem de tempo nas sessões.

Estado global gerenciado:

- `Sessao sessoes[MAX_SESSOES]` — array de sessões
- `int total_sessoes` — quantas existem
- `int proximo_id` — próximo ID a atribuir
- `int hora_simulada, minuto_simulada` — relógio do simulador

Funções públicas principais:

Função	O que faz
<code>sessao_inicializar_sistema</code>	Zera contadores e inicia o relógio em 09:00
<code>sessao_criar</code>	Adiciona sessão ao array, emite OCPP de início
<code>sessao_encerrar</code>	Marca como FINALIZADA , emite OCPP de fim
<code>sessao_listar</code>	Imprime tabela com todas as sessões
<code>sessao_imprimir_detalhe</code>	Mostra dados completos de uma sessão
<code>sessao_avancar_tempo</code>	Avança o relógio, atualiza kWh e custo de cada sessão ativa, emite MeterValues
<code>demanda_total_atual</code>	Soma a potência atual de todas as sessões CARREGANDO

3.3 Módulo **demanda** — controle de demanda inteligente

Responsabilidade: redistribuir potência entre sessões ativas respeitando o limite contratado do edifício e a prioridade de cada cliente.

Algoritmo: descrito em detalhe na Seção 4.1 deste documento.

Funções públicas:

Função	O que faz
<code>demanda_aplicar</code>	Executa a redistribuição. Retorna 1 se houve ajuste, 0 se houve folga
<code>demanda_imprimir_status</code>	Mostra ocupação atual separada por prioridade

O módulo é acionado automaticamente após `sessao_criar` e `sessao_encerrar`, garantindo que o estado do sistema seja sempre consistente.

3.4 Módulo **tarifa** — preço dinâmico

Responsabilidade: calcular o preço por kWh em função de três variáveis e aplicar esse preço ao custo acumulado das sessões.

Algoritmo: descrito na Seção 4.2.

Funções públicas:

Função	O que faz
<code>tarifa_calcular</code>	Função pura — retorna preço/kWh dadas as variáveis de contexto
<code>tarifa_recalcular_sesoes</code>	Atualiza <code>custo_acumulado</code> de todas as sessões com a tarifa atual
<code>tarifa_explicar</code>	Imprime decomposição dos multiplicadores no console
<code>nome_faixa_horaria</code>	Retorna "PONTA", "FORA DE PONTA", etc.
<code>nome_faixa_ocupacao</code>	Retorna "PICO", "MODERADA", "FOLGA"

3.5 Módulo `ocpp` — simulação do protocolo

Responsabilidade: simular o tráfego OCPP 1.6-J entre o Charge Point e o Central System Management Software (CSMS), conforme especificação oficial da Open Charge Alliance.

Estado gerenciado:

- `MensagemOCPP mensagens_ocpp[MAX_MENSAGENS_OCPP]` — buffer de mensagens trafegadas
- `int total_mensagens_ocpp` — quantas existem
- Contador de `messageId` interno

Funções públicas:

Função	Action OCPP correspondente
<code>ocpp_emit_boot_notification</code>	<code>BootNotification</code> — registro inicial do carregador
<code>ocpp_emit_heartbeat</code>	<code>Heartbeat</code> — keep-alive periódico
<code>ocpp_emit_authorize</code>	<code>Authorize</code> — validação do idTag (cartão/app)
<code>ocpp_emit_start_transaction</code>	<code>StartTransaction</code> — início da sessão
<code>ocpp_emit_meter_values</code>	<code>MeterValues</code> — leitura periódica do medidor
<code>ocpp_emit_stop_transaction</code>	<code>StopTransaction</code> — fim da sessão
<code>ocpp_emit_status_notification</code>	<code>StatusNotification</code> — mudança de estado do connector
<code>ocpp_console_imprimir</code>	Mostra mensagens do buffer

Detalhes da estrutura das mensagens estão na Seção 5.

3.6 Módulo `relatorio` — KPIs consolidados

Responsabilidade: consolidar os dados da simulação em um relatório operacional auditável, exibível tanto no console quanto em arquivo `.txt`.

Funções públicas:

Função	O que faz
<code>relatorio_imprimir_tela</code>	Mostra o resumo no terminal
<code>relatorio gerar_arquivo</code>	Cria arquivo <code>relatorios/relatorio_HHMM.txt</code>

Conteúdo do relatório (6 blocos):

1. **Panorama operacional** — sessões por estado, modelo HCA, prioridade
2. **Energia entregue** — kWh total, médio, máximo; ocupação do edifício
3. **Performance financeira** — receita total, ticket médio, receita por modelo
4. **Contexto tarifário** — multiplicadores em vigor, tarifa por modelo
5. **Tráfego OCPP** — contagem de mensagens por action
6. **Detalhamento por sessão** — tabela completa

3.7 `main.c` — orquestrador

Responsabilidade: apresentar o menu interativo, validar entradas e dispatchar para os módulos.

O `main` não contém lógica de negócio — apenas leitura de input e chamadas a funções dos módulos. Toda a inteligência vive nos módulos. Esse padrão (controlador "burro" + serviços com lógica) é o paradigma do design modular limpo.

4. Algoritmos centrais

4.1 Algoritmo de controle de demanda por prioridade

Quando a demanda total das sessões ativas excede a `DEMANDA_CONTRATADA` do edifício (75 kW por padrão), o sistema precisa decidir quem recebe quanta potência. Nossa implementação usa **triagem por prioridade**, semelhante ao que ocorre em sistemas reais de orquestração de carregamento.

Estados de sessão considerados

O algoritmo considera elegíveis as sessões em estado `CARREGANDO` ou `PAUSADA_DEMANDA`. Sessões `FINALIZADA`, `ERRO` ou `AGUARDANDO_AUTORIZACAO` não consomem potência e são ignoradas.

Tiers de prioridade

Prioridade	Tipo	Política
3	Emergência	Atendida primeiro, recebe a potência nominal
2	Frota corporativa	Atendida em seguida, recebe nominal se houver orçamento
1	Usuário normal	Divide o que sobrar proporcionalmente à potência nominal

Fluxo do algoritmo

```

1. Calcular pedido_bruto = soma das potencias nominais das sessoes elegiveis

2. Se pedido_bruto <= DEMANDA_CONTRATADA:
    cada sessao recebe sua potencia nominal
    FIM (folga - sem throttling)

3. Caso contrario (sobrecarga):
    orcamento = DEMANDA_CONTRATADA

    para cada prioridade p em [3, 2, 1]:
        pedido_p = soma dos nominais das sessoes da prioridade p

        se pedido_p <= orcamento:
            cada sessao da prioridade p recebe seu nominal
            orcamento = orcamento - pedido_p
        senao:
            fator = orcamento / pedido_p
            cada sessao da prioridade p recebe nominal * fator
            orcamento = 0

4. Para cada sessao com potencia_atual_kw < MIN_POTENCIA_CARGA:
    estado = PAUSADA_DEMANDA
    emite OCPP StatusNotification "SuspendedEVSE"

5. Para cada sessao previamente pausada que volta acima do limiar:
    estado = CARREGANDO
    emite OCPP StatusNotification "Charging"

```

Exemplo numérico

Cenário: 5 sessões **GW22K-HCA-20** (22 kW cada), prioridades 3, 2, 1, 1, 1.

- Pedido bruto: $5 \times 22 = \mathbf{110 \text{ kW}}$
- Orçamento: **75 kW**
- Sobrecarga de 35 kW

Passo	Prioridade	Pedido	Recebe	Orçamento restante
1	3 (1 sessão)	22 kW	22 kW	53 kW
2	2 (1 sessão)	22 kW	22 kW	31 kW
3	1 (3 sessões)	66 kW	fator = $31/66 = 0.470$	0 kW

Cada sessão de prioridade 1 recebe: $22 \times 0.470 = \mathbf{10.33 \text{ kW}}$

Verificação: $22 + 22 + 10.33 \times 3 = 74.99 \text{ kW} \approx 75 \text{ kW}$ (orçamento respeitado).

Todas as sessões permanecem em **CARREGANDO** porque $10.33 \text{ kW} > 1.4 \text{ kW}$ (limiar IEC 61851).

Limiar IEC 61851

A norma IEC 61851 estabelece que carregadores não devem operar abaixo de aproximadamente 1.4 kW (equivalente a 6A em 230V). Sessões cuja alocação caia abaixo desse limiar são automaticamente pausadas (**PAUSADA_DEMANDA**), e voltam a carregar quando o orçamento permitir. Essa lógica replica o comportamento de hardware real.

4.2 Tarifação dinâmica multi-variável

A tarifa por kWh é calculada como produto de quatro componentes:

$$\text{tarifa} = \text{TARIFA_BASE} \times \text{mult_horario} \times \text{mult_demanda} \times \text{mult_modelo}$$

Componente 1: TARIFA_BASE

Valor de referência de R\$ 0.85/kWh, inspirado em tarifas residenciais brasileiras típicas.

Componente 2: multiplicador de horário

Inspirado na **estrutura tarifária branca** definida pela ANEEL na Resolução Normativa 733/2016, que aplica preços diferentes ao longo do dia para incentivar o uso da rede fora dos horários de pico.

Faixa	Horas	Multiplicador
PONTA	18h–20h	× 1.50
INTERMEDIARIO	17h, 21h	× 1.20
PADRAO	demais horários do dia	× 1.00
FORA DE PONTA	22h–05h	× 0.70

Componente 3: multiplicador de demanda

Aplica sobretaxa progressiva conforme a ocupação do edifício, lógica semelhante ao *surge pricing* de aplicativos de mobilidade. O objetivo é desincentivar novas sessões quando o sistema está próximo do limite contratado.

Faixa	Ocupação	Multiplicador
PICO	> 80%	× 1.30
MODERADA	50% – 80%	× 1.10
FOLGA	< 50%	× 1.00

Componente 4: multiplicador de modelo

Reflete a lógica comercial de cobrar premium por velocidade de recarga — o cliente do **GW22K** está pagando pelo "tempo economizado" que esse modelo oferece.

Modelo	Multiplicador
GW7K-HCA-20	× 1.00
GW11K-HCA-20	× 1.10
GW22K-HCA-20	× 1.20

Exemplo numérico

Cenário: Recarga em um **GW22K-HCA-20** às 18h30, com edifício a 100% de ocupação.

```
tarifa = 0.85 × 1.50 (PONTA) × 1.30 (PICO) × 1.20 (GW22K)
        = 0.85 × 2.34
        = 1.989 R$/kWh
```

Comparado à tarifa base (R\$ 0.85), essa sessão paga **134% a mais** por kWh — refletindo corretamente o custo marginal real (energia mais cara na rede + congestionamento no edifício + premium pelo equipamento).

5. Simulação do protocolo OCPP 1.6

5.1 O que é o OCPP

O **OCPP (Open Charge Point Protocol)** é o protocolo padrão da indústria para comunicação entre carregadores de veículos elétricos (**Charge Point**) e sistemas centrais de gerenciamento (**Central System Management Software** — CSMS). Mantido pela Open Charge Alliance, é adotado por GoodWe, ABB, Schneider, EVBox, ChargePoint e praticamente todos os principais fabricantes.

A versão 1.6, lançada em 2015 e ainda hoje a mais difundida, usa **JSON sobre WebSocket** como camada de transporte (variante chamada **OCPP-J 1.6**).

5.2 Formato das mensagens

Toda mensagem OCPP-J é um array JSON com a seguinte estrutura:

```
[ tipo, messageId, action, payload ]      <- Call (requisicao)
[ tipo, messageId, payload ]              <- CallResult (resposta)
[ tipo, messageId, errorCode, errorDesc ] <- CallError (erro)
```

Onde:

- **tipo** é um inteiro: **2** para Call, **3** para CallResult, **4** para CallError
- **messageId** é uma string única que correlaciona Call com CallResult
- **action** é o nome da operação (ex: "**BootNotification**")
- **payload** é um objeto JSON com os dados da operação

5.3 Actions implementadas

Nossa simulação cobre **sete actions** do subset essencial do OCPP 1.6:

Action	Direção	Quando é emitida
BootNotification	CP → CSMS	Inicialização do sistema
Heartbeat	CP → CSMS	A cada ciclo de avanço de tempo
Authorize	CP → CSMS	Quando o usuário inicia uma sessão
StartTransaction	CP → CSMS	Após autorização aceita
MeterValues	CP → CSMS	Periodicamente durante a recarga
StopTransaction	CP → CSMS	Quando a sessão é encerrada
StatusNotification	CP → CSMS	Mudança de estado do connector

5.4 Exemplo de mensagem real emitida

Trecho real da saída do sistema, mostrando uma mensagem **MeterValues** emitida durante uma sessão de recarga:

```
[OCPP 2026-06-12T18:30:00Z >] MeterValues | [2, "msg-00012", "MeterValues",
  {"connectorId":1,"transactionId":1,"meterValue":[{"timestamp":"2026-06-
12T18:30:00Z",
  "sampledValue":
  [{"value":"11.00","measurand":"Energy.Active.Import.Register","unit":"kWh"},

  {"value":"22.00","measurand":"Power.Active.Import","unit":"kW"}]}]]]
[OCPP 2026-06-12T18:30:00Z <] MeterValues reply | [3, "msg-00012", {}]
```

Observe os elementos que tornam essa mensagem **estruturalmente correta** em relação ao OCPP 1.6:

- Formato de array com tipo **[2, ...]** para Call e **[3, ...]** para CallResult
- **messageId** correlacionado entre Call e Result
- Timestamps em ISO 8601 UTC
- **measurand** no vocabulário do protocolo (**Energy.Active.Import.Register**, **Power.Active.Import**)
- Direção indicada pelas setas **>** (CP→CSMS) e **<** (CSMS→CP)

5.5 Fluxo típico de uma sessão completa

Sequência das mensagens trafegadas quando uma sessão de recarga executa o ciclo completo:

1. BootNotification (na inicializacao do sistema)
2. Authorize (operador cria sessao via menu)
3. StartTransaction (transacao registrada no CSMS)
4. StatusNotification (connector vai para "Charging")

5. MeterValues (a cada avanço de tempo)
6. Heartbeat (entre ciclos)
... repete ...
7. StopTransaction (operador encerra sessao)
8. StatusNotification (connector volta para "Available")

6. Compilação e execução

6.1 Pré-requisitos

- **Sistema operacional:** Windows 10/11, Linux ou macOS
- **Compilador:** GCC (MinGW-w64 no Windows, gcc nativo no Linux/macOS)
- **Editor:** Visual Studio Code com extensão C/C++ da Microsoft

6.2 Instalação do GCC no Windows

1. Baixar e instalar o MSYS2 a partir de [msys2.org](https://www.msys2.org)
2. Abrir o terminal "MSYS2 MINGW64" e executar:

```
pacman -S mingw-w64-x86_64-gcc
```

3. Adicionar `C:\msys64\mingw64\bin` à variável de ambiente PATH
4. Reiniciar o VS Code completamente
5. Verificar com `gcc --version` no terminal

6.3 Compilação via VS Code

Com o projeto aberto no VS Code, pressionar `Ctrl+Shift+B`. A tarefa configurada em `.vscode/tasks.json` compila todos os seis módulos e gera o executável `chargegrid.exe` (Windows) ou `chargegrid` (Linux/macOS) na raiz do projeto.

6.4 Compilação manual

```
gcc -Wall -Wextra -std=c11 -o chargegrid \  
src/main.c src/sessao.c src/demanda.c src/tarifa.c \  
src/ocpp.c src/relatorio.c
```

As flags `-Wall` `-Wextra` ativam todos os avisos do compilador, e `-std=c11` força o padrão C11. O projeto compila sem nenhum warning.

6.5 Execução

No terminal, na raiz do projeto:

```
.\chargegrid.exe      (Windows)
./chargegrid          (Linux/macOS)
```

O sistema abre o menu interativo. Digite 0 para encerrar.

7. Cenários de teste

Cenário 1: fluxo básico

Objetivo: validar criação, listagem e encerramento de sessões.

Passo	Entrada	Resultado esperado
1	1 → ABC1D23 → 2 → 9 → 30 → 1	Sessão 1 criada, modelo GW11K, OCPP Authorize + StartTransaction + StatusNotification emitidos
2	2	Tabela mostra a sessão 1 com 0 kWh, estado CARREGANDO
3	5 → 1 → 60	Avança 60 min, OCPP MeterValues + Heartbeat emitidos
4	3 → 1	Detalhe mostra 11.00 kWh entregues, R\$ 9.35 cobrados
5	4 → 1	Sessão finalizada, OCPP StopTransaction emitido

Cenário 2: sobrecarga e triagem por prioridade

Objetivo: validar o algoritmo de controle de demanda.

Passo	Entrada	Resultado esperado
1	1 × 5 (criar 5 sessões GW22K, prioridades 1, 1, 1, 2, 3)	110 kW pedidos contra 75 kW contratados
2	Após a 4ª sessão	Aparece "SOBRECARGA - aplicando triagem"
3	Após a 5ª sessão	Triagem mostra: prio 3 → 22 kW, prio 2 → 22 kW, prio 1 → 10.33 kW cada

Cenário 3: tarifa de ponta

Objetivo: demonstrar a tarifação dinâmica em horário de ponta.

Passo	Entrada	Resultado esperado
1	5 → 2 → 18 → 30	Relógio salta para 18:30 (faixa PONTA)
2	7 → 1	Decomposição mostra mult. horário × 1.50, modelos com preços inflados
3	1 → ABC1D23 → 3 → 18 → 30 → 1	Sessão GW22K criada

Passo	Entrada	Resultado esperado
4	5 → 1 → 30	Avança 30 min
5	3 → 1	Custo acumulado: 11 kWh × R\$ 1.989/kWh = R\$ 21.88

Cenário 4: relatório completo

Objetivo: validar geração de arquivo `.txt`.

Passo	Entrada	Resultado esperado
1	Executar os Cenários 1, 2 ou 3	Acumular sessões e mensagens OCPP
2	9 → 2	Gera <code>relatorios/relatorio_HHMM.txt</code>
3	Abrir o arquivo em qualquer editor	Mostra os 6 blocos de KPIs com números consistentes

8. Mapeamento com a rubrica

Tabela de auditoria explícita: para cada critério da rubrica do Sprint 2, onde o código entrega o requisito e qual pontuação esperamos.

Item	Critério	Pontuação alvo	Onde está no código
1	Gerenciamento de múltiplas sessões	20/20	<code>sessao.h/sessao.c</code> — struct <code>Sessao</code> , array <code>sessoes[MAX_SESSOES]</code> , máquina de 6 estados, lifecycle completo
2	Controle de demanda de energia	20/20	<code>demanda.c</code> — algoritmo de load balancing por prioridade (Seção 4.1), limiar IEC 61851, redistribuição proporcional
3	Lógica de tarifação dinâmica	15/15	<code>tarifa.c</code> — três variáveis combinadas (horário × demanda × modelo, Seção 4.2), inspiração ANEEL
4	Simulação de integração OCPP/MODBUS	15/15	<code>ocpp.c</code> — protocolo OCPP 1.6-J real, sete actions, formato JSON estruturado, request/response correlacionado por <code>messageId</code>
5	Estrutura lógica do sistema	10/10	Seis módulos <code>.c/.h</code> separados, <code>tipos.h</code> compartilhado, dependências sem ciclos
6	Interatividade (menu e fluxo)	10/10	<code>main.c</code> — menu com sub-opções, validação de input em toda leitura, cabeçalho com relógio e status
7	Qualidade geral e robustez	10/10	Sem warnings (<code>-Wall -Wextra</code>), validação de buffers (<code>strncpy</code>), defesa contra divisão por zero, sem leaks

Total alvo: 100/100.

9. Limitações conhecidas

Em respeito à integridade técnica, listamos abaixo o que o sistema **não** faz, e por quê:

1. **Sem rede real** — o OCPP é simulado em memória. Em produção, as mensagens trafegariam via WebSocket Secure (WSS) com TLS 1.2+, e o `messageId` seria um UUID gerado por biblioteca dedicada.
2. **Sem persistência** — quando o programa encerra, todo o estado é perdido. Em produção, sessões e mensagens seriam persistidas em banco de dados (PostgreSQL, normalmente) com replicação.
3. **Sem concorrência** — o sistema é single-threaded. Em produção, cada conexão de carregador teria uma thread/coroutine dedicada, e o estado compartilhado seria protegido por mutexes ou message queues.
4. **Relógio simulado, não real** — o tempo no sistema é definido pelo operador via menu. Em produção, o relógio do CSMS seria sincronizado via NTP e cada mensagem teria timestamp real.
5. **Boundary crossing de tarifa não é granular** — se uma sessão atravessa a fronteira de 18h (PADRÃO → PONTA) durante um intervalo de avanço de tempo, todo o intervalo é cobrado na tarifa do relógio final. Em produção, o cálculo seria minuto a minuto.
6. **Sem MODBUS** — implementamos apenas OCPP. O MODBUS seria usado para comunicação direta com o hardware interno do carregador (medidor de energia, contator), e por escolha de escopo focamos no OCPP que é a interface externa visível ao operador.

Essas limitações são propositais e adequadas ao escopo de uma simulação acadêmica de primeiro ano. Elas estão documentadas aqui para sinalizar consciência de engenharia ao avaliador.

10. Referências

Protocolos e normas

1. **Open Charge Alliance.** *OCPP 1.6 Specification*. 2015. Disponível em: <https://www.openchargealliance.org/protocols/ocpp-16/>
2. **ANEEL.** *Resolução Normativa Nº 733/2016 — Estrutura Tarifária Branca*. Brasília, 2016.
3. **IEC.** *IEC 61851-1: Electric vehicle conductive charging system — Part 1: General requirements*. 4ª edição. International Electrotechnical Commission, 2017.

Documentação técnica

4. **GoodWe Brasil.** *ChargeGrid Intelligence — Linha HCA G2*. Disponível em: <https://en.goodwe.com/ev-chargers>
5. **Kernighan, B. W.; Ritchie, D. M.** *The C Programming Language*. 2ª edição. Prentice Hall, 1988.
6. **ISO/IEC.** *ISO/IEC 9899:2011 — Programming languages — C (C11 standard)*. 2011.

Recursos didáticos

7. **Microsoft.** *C/C++ for Visual Studio Code — Get Started*. Disponível em: <https://code.visualstudio.com/docs/cpp/config-mingw>
8. **GCC Manual.** *GCC Online Documentation*. Disponível em: <https://gcc.gnu.org/onlinedocs/>

Anexo A — Tabela de comandos do menu

Opção	O que faz
1	Inicia nova sessão de recarga (solicita placa, modelo, prioridade)
2	Lista todas as sessões em formato tabular
3	Mostra detalhes completos de uma sessão específica
4	Encerra uma sessão (libera orçamento de potência)
5	Tempo simulado (sub: avançar N minutos, saltar para HH:MM)
6	Controle de demanda (sub: aplicar redistribuição, ver status)
7	Tarifa dinâmica (sub: ver decomposição, recalcular custos)
8	Console OCPP (sub: ver todas, ver últimas 10, limpar, toggle verbose)
9	Relatório (sub: imprimir na tela, gerar arquivo <code>.txt</code>)
0	Encerra o programa

Anexo B — Roteiro de defesa oral

Pontos que o grupo deve estar preparado para discutir:

1. **Por que array fixo em vez de lista encadeada?** Resposta na Seção 2.4.
 2. **Como funciona o controle de demanda quando 5 carros chegam?** Resposta na Seção 4.1, com exemplo numérico.
 3. **Quais as três variáveis da tarifa dinâmica?** Horário, demanda do edifício, modelo do carregador. Seção 4.2.
 4. **O que é OCPP e por que vocês escolheram esse protocolo?** Padrão de indústria, adoção universal pela GoodWe e concorrentes. Seção 5.1.
 5. **Como vocês garantiram que a simulação OCPP é estruturalmente correta?** Formato JSON-array com tipo, messageld, action, payload. Vocabulário de measurand do OCPP 1.6. Seção 5.2.
 6. **Por que vocês separaram em seis módulos?** Princípio da responsabilidade única — cada arquivo cuida de um aspecto. Facilita manutenção e teste. Seção 2.4.
 7. **O que aconteceria se aparecesse uma 21ª sessão?** O sistema rejeitaria com mensagem "[erro] Capacidade maxima atingida". Limite de `MAX_SESSOES = 20`.
 8. **O que é o limiar IEC 61851 e por que ele importa?** 1.4 kW (6A em 230V) é o mínimo abaixo do qual carregadores reais não operam. Implementado em `ajustar_estados_pos_alocacao` em `demanda.c`. Seção 4.1.
-

Documento gerado para a Sprint 2 da disciplina Data Structure and Algorithms, EV Challenge 2026, FIAP × GoodWe.